

A Closer Look

React Server Components & Server Actions

Easy Guide

Contents

1	Understanding React Server Components	2
1.1	What are Server Components?	2
1.1.1	Traditional React Components	2
1.1.2	Server Components	2
1.2	Key Difference	3
1.3	Why Server Components?	3
1.4	Example: Blog Post	3
2	Changing Server to Client Components	5
2.1	When to Add 'use client'	5
2.2	Why Make a Component Client-Side?	5
2.3	Default is Server	6
2.4	The 'use client' Boundary	6
3	Combining Server and Client Components	7
3.1	The Best Approach	7
3.2	Pattern: Server Fetch, Client Display	7
3.3	Pattern: Separate Concerns	8
3.4	Benefits of This Approach	9
4	Fetching Data with RSCs	10
4.1	Direct Database Queries	10
4.2	API Calls	10
4.3	Dynamic Parameters	11
4.4	Multiple Data Sources	11
5	Submitting Data with Server Actions	13
5.1	What are Server Actions?	13
5.2	Creating a Server Action	13
5.3	Using in Client Component	13
5.4	Form Action Syntax	14
5.5	Server Action with Validation	14
6	Using the use() Hook for Promises	16
6.1	What is use()?	16
6.2	The Problem	16
6.3	The Solution: use() Hook	16
6.4	Why use use()?	17

7	The use() Hook In Action	18
7.1	Real Example: User Profile	18
7.2	Multiple Promises	19
7.3	Combining use() and Suspense	20
8	Handling Errors	21
8.1	Try/Catch in Server Components	21
8.2	Error Boundaries	21
8.3	Server Action Errors	21
8.4	Async Errors with use()	22
8.5	Validation Errors	23

Chapter 1

Understanding React Server Components

1.1 What are Server Components?

Components that run on the server only. Not sent to the browser.

1.1.1 Traditional React Components

```
// All runs in browser
export default function Post() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('/api/post')
      .then(r => r.json())
      .then(d => setData(d));
  }, []);

  return <div>{data?.title}</div>;
}
```

```
// Downloaded to browser:
// - Component code
// - useState hook
// - useEffect hook
// - Fetch logic
// - All dependencies
```

Large bundle! Lots of JavaScript sent to browser.

1.1.2 Server Components

```
// Runs on server only
export default async function Post() {
  const data = await fetch('http://localhost:3000/api/post')
```

```

    .then(r => r.json());

    return <div>{data.title}</div>;
  }

// Downloaded to browser:
// - Only HTML result: <div>Post Title</div>
// - No JavaScript
// - No component code

```

Tiny browser bundle! Much faster!

1.2 Key Difference

Feature	Client Component	Server Component
Runs where	Browser	Server
Can use hooks	Yes	No
Can fetch data	Yes (with effect)	Yes (directly)
Bundle size	Large	Tiny
Interactivity	Yes	No

1.3 Why Server Components?

1. **Smaller bundles** - Less JavaScript to download
2. **Faster loading** - No rendering delay
3. **Direct DB access** - No API needed
4. **Security** - Secrets stay on server
5. **Better performance** - Server renders HTML

1.4 Example: Blog Post

Old way (Client Component):

- User downloads component code (5KB)
- User downloads React library (35KB)
- User downloads fetch logic (2KB)
- Browser makes API request
- Browser renders component
- Total: 42KB + network delay

New way (Server Component):

- Server fetches data from database
- Server renders HTML
- Server sends HTML to browser (8KB)
- Browser displays instantly
- Total: 8KB + no delay

Result: 5x smaller, instant display!

Chapter 2

Changing Server to Client Components

2.1 When to Add 'use client'

Add 'use client' directive to make a component run in browser:

```
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

2.2 Why Make a Component Client-Side?

Use client components when you need:

- **useState** - State management
- **useEffect** - Side effects
- **useContext** - Context
- **Event handlers** - onClick, onChange, etc.
- **Browser APIs** - localStorage, window, etc.

2.3 Default is Server

```
// This is a SERVER component (default)
export default async function Posts() {
  const posts = await fetchPosts();
  return <div>{/* render posts */</div>;
}

// This is a CLIENT component (need 'use client')
'use client';
export default function PostForm() {
  const [title, setTitle] = useState('');
  return <input onChange={e => setTitle(e.target.value)} />;
}
```

2.4 The 'use client' Boundary

Once you add 'use client', that component and all its children become client components.

```
// components/Layout.js (Server)
import { ClientComponent } from './ClientComponent';

export default function Layout() {
  return (
    <div>
      <Header />   {/* Server component */}
      <ClientComponent />   {/* Client component */}
      <Footer />   {/* Server component */}
    </div>
  );
}

// components/ClientComponent.js
'use client';

export function ClientComponent() {
  // This and everything here is client-side
  return <div>Client content</div>;
}
```

Principle: Server by default, client only when needed!

Chapter 3

Combining Server and Client Components

3.1 The Best Approach

Use server components for data, client components for interactivity.

3.2 Pattern: Server Fetch, Client Display

```
// app/posts/page.js (Server)
import PostCard from '@components/PostCard';

async function getPosts() {
  const posts = await db.post.findMany();
  return posts;
}

export default async function PostsPage() {
  const posts = await getPosts();

  return (
    <div>
      <h1>Posts</h1>
      {posts.map(post => (
        <PostCard key={post.id} post={post} />
      ))}
    </div>
  );
}

// components/PostCard.js (Client)
'use client';

import { useState } from 'react';
```

```
export default function PostCard({ post }) {
  const [liked, setLiked] = useState(false);

  return (
    <div>
      <h2>{post.title}</h2>
      <p>{post.content}</p>
      <button onClick={() => setLiked(!liked)}>
        {liked ? ' Liked' : ' Like'}
      </button>
    </div>
  );
}
```

3.3 Pattern: Separate Concerns

```
// app/dashboard/page.js (Server - fetches data)
import StatCard from '@components/StatCard';
import Chart from '@components/Chart';

async function getStats() {
  return db.stats.findMany();
}

export default async function Dashboard() {
  const stats = await getStats();

  return (
    <div>
      <StatCard title="Users" value={stats.users} />
      <Chart data={stats} />
    </div>
  );
}

// components/Chart.js (Client - interactivity)
'use client';

import { useState } from 'react';

export default function Chart({ data }) {
  const [view, setView] = useState('monthly');

  return (
    <div>
      <select value={view} onChange={(e) => setView(e.target.value)}>
        <option>monthly</option>
      </select>
    </div>
  );
}
```

```
        <option>yearly</option>
      </select>
      {/* Render chart based on view */}
    </div>
  );
}
```

3.4 Benefits of This Approach

1. **Smaller bundles** - Server components not sent to browser
2. **Direct DB access** - Fetch in server component
3. **Interactivity** - Client component handles state
4. **Security** - DB credentials stay on server
5. **Performance** - Less code to download

Best Practice: Keep components on server as much as possible!

Chapter 4

Fetching Data with RSCs

4.1 Direct Database Queries

```
// app/users/page.js (Server Component)
import db from '@lib/database';

async function getUsers() {
  // Direct database access - no API needed!
  const users = await db.user.findMany();
  return users;
}

export default async function Users() {
  const users = await getUsers();

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

4.2 API Calls

```
// app/posts/page.js (Server Component)
async function getPosts() {
  const response = await fetch('https://api.example.com/posts', {
    next: { revalidate: 60 } // Cache for 60 seconds
  });
  return response.json();
}

export default async function Posts() {
```

```

const posts = await getPosts();

return (
  <div>
    {posts.map(post => (
      <div key={post.id}>{post.title}</div>
    ))}
  </div>
);
}

```

4.3 Dynamic Parameters

```

// app/blog/[slug]/page.js
async function getPost(slug) {
  const post = await db.post.findUnique({
    where: { slug }
  });
  return post;
}

export default async function BlogPost({ params }) {
  const post = await getPost(params.slug);

  return (
    <article>
      <h1>{post.title}</h1>
      <p>{post.content}</p>
    </article>
  );
}

```

4.4 Multiple Data Sources

```

// app/dashboard/page.js
async function getUserData(userId) {
  return db.user.findUnique({ where: { id: userId } });
}

async function getUserPosts(userId) {
  return db.post.findMany({ where: { userId } });
}

async function getUserStats(userId) {
  return db.stats.findUnique({ where: { userId } });
}

```

```
export default async function Dashboard() {
  // Fetch all in parallel
  const [user, posts, stats] = await Promise.all([
    getUserData(userId),
    getUserPosts(userId),
    getUserStats(userId)
  ]);

  return (
    <div>
      <h1>{user.name}</h1>
      <p>Posts: {posts.length}</p>
      <p>Views: {stats.views}</p>
    </div>
  );
}
```

Advantage: No API needed. Direct DB queries!

Chapter 5

Submitting Data with Server Actions

5.1 What are Server Actions?

Functions that run on the server. Triggered from client forms.

5.2 Creating a Server Action

```
// app/actions.js
'use server';

export async function createPost(formData) {
  const title = formData.get('title');
  const content = formData.get('content');

  const post = await db.post.create({
    data: { title, content }
  });

  return post;
}
```

5.3 Using in Client Component

```
// app/components/PostForm.js
'use client';

import { createPost } from '@app/actions';

export default function PostForm() {
  async function handleSubmit(e) {
    e.preventDefault();
    const formData = new FormData(e.target);

    try {
      const post = await createPost(formData);
    }
  }
}
```



```

        console.log('Created:', post);
    } catch (error) {
        console.error('Error:', error);
    }
}

return (
  <form onSubmit={handleSubmit}>
    <input name="title" placeholder="Title" required />
    <textarea name="content" placeholder="Content" required />
    <button type="submit">Create</button>
  </form>
);
}

```

5.4 Form Action Syntax

```

// Simpler: use action prop
'use client';

import { createPost } from '@app/actions';

export default function PostForm() {
  return (
    <form action={createPost}>
      <input name="title" required />
      <textarea name="content" required />
      <button type="submit">Create</button>
    </form>
  );
}

```

5.5 Server Action with Validation

```

'use server';

export async function createPost(formData) {
  const title = formData.get('title');
  const content = formData.get('content');

  // Validation
  if (!title || title.length < 3) {
    throw new Error('Title must be at least 3 characters');
  }

  if (!content || content.length < 10) {
    throw new Error('Content must be at least 10 characters');
  }
}

```

```
}

// Save to database
const post = await db.post.create({
  data: { title, content }
});

// Revalidate cache
revalidatePath('/blog');

return post;
}
```

Magic: Call server functions directly from client forms!

Chapter 6

Using the use() Hook for Promises

6.1 What is use()?

Hook to resolve promises in client components.

6.2 The Problem

```
// app/page.js (Server Component)
async function getData() {
  return fetch('...').then(r => r.json());
}

// Can't pass async function to client component!
// This doesn't work:
export default async function Home() {
  const promise = getData();
  return <ClientComponent promise={promise} />;
}
```

6.3 The Solution: use() Hook

```
// app/components/ClientComponent.js
'use client';

import { use } from 'react';

export default function ClientComponent({ promise }) {
  // Resolve promise in client component
  const data = use(promise);

  return <div>{data.title}</div>;
}

// app/page.js (Server Component)
async function getData() {
```

```
    return fetch('...').then(r => r.json());
  }

export default async function Home() {
  const dataPromise = getData();

  return (
    <Suspense fallback={<p>Loading...</p>}>
      <ClientComponent promise={dataPromise} />
    </Suspense>
  );
}
```

6.4 Why use use()?

Pass async data from server to client component:

1. Server starts fetching
2. Passes promise to client
3. Client waits with Suspense
4. Client renders when ready
5. Faster than fetching in client

Use Case: Fetch on server, resolve in client!

Chapter 7

The use() Hook In Action

7.1 Real Example: User Profile

```
// app/components/UserProfile.js
'use client';

import { use } from 'react';

export default function UserProfile({ userPromise }) {
  const user = use(userPromise);

  return (
    <div>
      <h1>{user.name}</h1>
      <p>Email: {user.email}</p>
    </div>
  );
}

// app/users/[id]/page.js (Server)
import { Suspense } from 'react';
import UserProfile from '@components/UserProfile';

async function getUser(id) {
  const user = await db.user.findUnique({
    where: { id }
  });
  return user;
}

export default function UserPage({ params }) {
  const userPromise = getUser(params.id);

  return (
    <Suspense fallback={<p>Loading user...</p>}>
```

```

        <UserProfile userPromise={userPromise} />
      </Suspense>
    );
  }

```

7.2 Multiple Promises

```

// app/components/Dashboard.js
'use client';

import { use } from 'react';

export default function Dashboard({
  userPromise,
  postsPromise,
  statsPromise
}) {
  const user = use(userPromise);
  const posts = use(postsPromise);
  const stats = use(statsPromise);

  return (
    <div>
      <h1>{user.name}</h1>
      <p>Posts: {posts.length}</p>
      <p>Views: {stats.views}</p>
    </div>
  );
}

// app/dashboard/page.js (Server)
import { Suspense } from 'react';
import Dashboard from '@components/Dashboard';

async function getUser(userId) {
  return db.user.findUnique({ where: { id: userId } });
}

async function getPosts(userId) {
  return db.post.findMany({ where: { userId } });
}

async function getStats(userId) {
  return db.stats.findUnique({ where: { id: userId } });
}

export default function DashboardPage() {

```

```

const userPromise = getUser(currentUserId);
const postsPromise = getPosts(currentUserId);
const statsPromise = getStats(currentUserId);

return (
  <Suspense fallback={<p>Loading dashboard...</p>}>
    <Dashboard
      userPromise={userPromise}
      postsPromise={postsPromise}
      statsPromise={statsPromise}
    />
  </Suspense>
);
}

```

7.3 Combining use() and Suspense

```

// Fine-grained loading states
export default function Page() {
  const userPromise = getUser();
  const postsPromise = getPosts();

  return (
    <>
      <Suspense fallback={<p>Loading user...</p>}>
        <UserCard userPromise={userPromise} />
      </Suspense>

      <Suspense fallback={<p>Loading posts...</p>}>
        <PostsList postsPromise={postsPromise} />
      </Suspense>
    </>
  );
}

```

Benefit: Partial loading - show data as it arrives!

Chapter 8

Handling Errors

8.1 Try/Catch in Server Components

```
// app/posts/page.js
export default async function Posts() {
  try {
    const posts = await db.post.findMany();
    return <PostsList posts={posts} />;
  } catch (error) {
    return <div>Error loading posts: {error.message}</div>;
  }
}
```

8.2 Error Boundaries

For client components:

```
// app/error.js
'use client';

export default function Error({ error, reset }) {
  return (
    <div>
      <h2>Something went wrong!</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>
        Try again
      </button>
    </div>
  );
}
```

8.3 Server Action Errors

```
'use server';
```



```

export async function createPost(formData) {
  const title = formData.get('title');

  if (!title) {
    throw new Error('Title is required');
  }

  try {
    const post = await db.post.create({
      data: { title }
    });
    return { success: true, post };
  } catch (error) {
    throw new Error('Failed to create post');
  }
}

// Client component
'use client';

export default function PostForm() {
  async function handleSubmit(e) {
    e.preventDefault();
    const formData = new FormData(e.target);

    try {
      const result = await createPost(formData);
      console.log('Success:', result);
    } catch (error) {
      console.error('Error:', error.message);
      // Show error to user
    }
  }

  return <form onSubmit={handleSubmit}>{ /* ... */ }</form>;
}

```

8.4 Async Errors with use()

```

// app/components/UserProfile.js
'use client';

import { use } from 'react';

export default function UserProfile({ userPromise }) {
  try {

```

```

    const user = use(userPromise);
    return <div>{user.name}</div>;
  } catch (error) {
    return <p>Error loading user: {error.message}</p>;
  }
}

```

8.5 Validation Errors

```

'use server';

export async function updateUser(formData) {
  const email = formData.get('email');

  // Validate format
  if (!email.includes('@')) {
    throw new Error('Invalid email format');
  }

  // Check if exists
  const existing = await db.user.findUnique({
    where: { email }
  });

  if (existing) {
    throw new Error('Email already in use');
  }

  // Update
  return db.user.update({
    where: { id: userId },
    data: { email }
  });
}

```

Pattern: Validate server-side, catch errors on client!

Quick Summary

Key Concepts

- **Server Components** - Default, run on server, small bundles
- **Client Components** - Need 'use client', run in browser
- **Server Actions** - Functions that run on server, called from forms
- **use() Hook** - Resolve promises in client components
- **Suspense** - Show loading states while fetching

When to Use Server Components

- Fetch data from database
- Access environment variables/secrets
- Keep API keys hidden
- Reduce JavaScript sent to browser
- Render static content

When to Use Client Components

- Need useState, useEffect, hooks
- Need event handlers (onClick, onChange, etc.)
- Need browser APIs (localStorage, window, etc.)
- Need user interactivity
- Use context

Pattern: Combining Both

1. Server component fetches data
2. Server component renders client components
3. Client components handle interactivity
4. Minimal JavaScript sent to browser
5. Fast loading, interactive UI

Best Practices

1. **Default to server** - Only use 'use client' when needed
2. **Keep server logic on server** - Don't expose in client bundle
3. **Fetch early** - Fetch in server components
4. **Use Suspense** - Show loading states
5. **Handle errors** - Try/catch for failures
6. **Validate on server** - Never trust client

Performance Benefits

- Smaller bundle sizes (80-90% reduction possible)
- Faster initial load
- Direct database access (no API overhead)
- Better SEO (server renders HTML)
- Improved Time to Interactive

Common Mistakes to Avoid

- Using hooks in server components
- Making everything client components
- Not using Suspense for loading states
- Forgetting to handle errors
- Putting secrets in client component
- Not validating on server

File Organization

```
app/  
  page.js           (Server component - fetches data)  
  layout.js         (Server component - layout)  
  error.js          (Client component - error boundary)  
  actions.js        (Server actions)  
  components/  
    Button.js       (Client - interactivity)  
    Card.js         (Server - static content)  
    Form.js         (Client - form with action)
```

Server Components are the future of React - smaller bundles, faster apps, better performance!